

3. Interrupts

1. Welche Arten von Interrupts gibt es? Wodurch werden diese ausgelöst?

Interrupts sind externe, asynchrone Ausnahmen, die nicht direkt mit dem aktuellen Programmablauf (im Sinne der momentanen Instruktion) zu tun haben (s. 11.17).

Grob gesehen, können wir unterscheiden zwischen IO und Timer Interrupts.

Beispiele für IO:

- Eingaben wie Maus und Tastatur
- IO Prozesse wie Rückmeldung eines Speicherträgers über erfolgreichen Schreibprozess

2. Wie bzw. wann wird auf das Eintreten eines Interrupts reagiert? Wie unterscheiden sich die Arten hier? Auf welcher Betrachtungsebene (Mikroinstruktionen, Instruktionen, Programme, . . .) erfolgt dies?

Control detektiert Ausnahme und leitet Behandlung ein (11.22). Es wird je nach Ausnahme die Ausführung einer Ausnahmeroutine (Interrupt Service Routine, *ISR*) eingeleitet.

Die Arten von Interrupt (IO, Timer) können sich durch Signalart unterscheiden (z.B. sporadische IO Rückmeldung vs. periodischer Timer). Das Schema der Behandlung bleibt aber gleich.

Die Behandlung eines Interrupts erfolgt transparent und ist für das Programm nicht sichtbar – es vergeht lediglich Zeit zwischen zwei Instruktionen. Die *ISR* wird von Control durch Mikroinstruktionen angestoßen.

3. Was ergibt sich daraus für die Verantwortlichkeiten im Falle eines Interrupts? Gehen Sie zudem auf die Sinnhaftigkeit von Rückgabewerten ein.

Bei einem Aufruf einer *ISR* gibt es keinen “Aufrufenden” im klassischen Sinne, da der Aufruf nicht aus einem Programmablauf sondern von der Hardware selbst verursacht wird. Dementsprechend werden die Aufgaben des Aufrufers auf Control sowie die *ISR* selbst verteilt. Control stellt sicher, dass nach der Behandlung des Interrupts das Programm an der selben Stelle fortsetzen kann. Die *ISR* gewährleistet, dass der übrige State erhalten bleibt (Stack, Register).

Dementsprechend haben Interrupt-Routinen keinen Rückgabewert. Es gab keinen Aufrufenden im klassischen Sinne, also auch niemanden der einen Rückgabewert erwarten würde. Das Ziel ist es, das Programm nach dem Interrupt weiter laufen zu lassen, ‘als wäre nichts gewesen’.

4. Es gibt zudem die Möglichkeit, Interrupts zu sperren oder zu priorisieren, sodass die Behandlung nicht umgehend erfolgen muss. Welchen Nutzen könnte dies haben?

Priorisierung und Sperrung von Interrupts könnte vielfältige Anwendung haben:

- **Time-Sharing:** Ermöglicht Multitasking, indem Timer-Interrupts periodisch den Prozess wechseln (Folie 33-34).
- Interrupts Sperren während der Behandlung eines bestehenden Interrupts um Komplexität (nesting) zu vermeiden.
- Nach Interrupt-Quellen unterscheiden – und z.B. zeitkritische Interrupts zuerst behandeln.
- Kritische Programmabläufe ohne Unterbrechung, ‘atomar’, durchlaufen lassen.
- Bei knappen Ressourcen dafür sorgen, dass laufende Programme weiter laufen.

5. Gehen Sie von Interruptbehandlung anhand einer Sprungtabelle aus, von welcher in die richtige Interrupt Service Routine gesprungen wird.

a) Angenommen, die Sprungtabelle befände sich ohne Schreibschutz im Hauptspeicher. Welches Sicherheitsrisiko ergibt sich daraus? Welche Differenzierung wird nötig?

Risiko: Ein Angreifer könnte die Sprungtabelle manipulieren, sodass normaler Code die Kontrolle übernimmt, aber mit erhöhten Rechten läuft (Folie 36).

Differenzierung (Folie 37): Es ist eine "Zwei-Klassen-Gesellschaft" im Prozessor nötig:

1. **Normaler Modus (User Mode):** Eingeschränkte Rechte, kein Schreibzugriff auf Interrupt-Tabelle.
2. **Privilegierter Modus (Supervisor/Kernel Mode):** Darf alles.

Der Wechsel erfolgt automatisch bei einer Ausnahme (Folie 39).

Das Risiko ist privilege escalation. Ein Angreifer könnte durch Veränderung in der Sprungtabelle den Kontrollfluss so umleiten, dass von ihm gesteuerter Code als ISR ausgeführt wird. Da die CPU beim Eintritt einer Ausnahme in den privilegierten Modus wechselt, hätte dieser Code Zugriff auf systemkritische Ressourcen.

Es bleibt sicherzustellen, dass die Sprungtabelle nur zu geprüftem, vertrauenswürdigen Code springt. Es wird also nötig zu differenzieren zwischen systemkritischen (privilegierten) und user-space Operationen. So wird verhindert, dass beliebiger Code in geschützte Speicheradressen schreiben darf.

b) Stellen Sie den Prozess der Ausnahmebehandlung auf eine Ihres Erachtens geeignete Weise dar; schematisch, textuell, . . .

s. 11.28

1. Ausnahme passiert
2. Control detektiert Ausnahme. Kontextsicherung: PC (Fortsetzungsadresse).
3. Passende ISR wird anhand Sprungtabelle herausgesucht.
4. Sprung zur ISR. Kontextsicherung: Register, Stack.
5. Ausführung der ISR im privilegierten Modus.
6. ISR beendet: Kontextwiederherstellung. Rückkehr in nicht-privilegierten Modus.
7. Rücksprung zum gespeicherten PC-Wert.

c) Wie endet die Behandlung einer Ausnahme? Was muss der Prozessor hier leisten?

Es passieren zwei Dinge: privilege de-escalation und Rückkehr zum normalen Programmablauf. Es wird in den Normal-Modus gewechselt, nachdem die ISR im privilegierten Modus ausgeführt wurde (sret). Für die Rückkehr zum normalen Programmablauf stellt der Prozessor sicher, dass die Laufzeitumgebung vollständig auf den Zustand vor dem Eintreten der Ausnahme zurückgesetzt ist. – Dies beinhaltet die Wiederherstellung des Stacks sowie der Register. Das Programm kann dann mithilfe der gespeicherten Fortsetzungsadresse ab der nächsten auszuführenden Instruktion fortfahren.